

DAY 2: CODING EXERCISE

Build a Bitcoin Explorer in Go!

- ★ Show blockchain info
- ★ Check wallet balance
- ★ List recent transactions
- ★ Decode a transaction
- ★ Display block details

Files: `bootcamp-code/day2/go/starter/`

HOW APPS TALK TO BITCOIN

```
Your App (Go) ↔ Bitcoin Core (bitcoind)
                JSON-RPC
                Port 18443
```

JSON-RPC = Send JSON commands, get JSON responses

Same commands as `bitcoin-cli` but programmatic!

JSON-RPC REQUEST

```
{  
  "jsonrpc": "1.0",  
  "id": "explorer",  
  "method": "getblockchaininfo",  
  "params": []  
}
```

Response:

```
{  
  "result": {"chain": "regtest", "blocks": 101},  
  "error": null  
}
```

JUST THE STANDARD LIBRARY

No dependencies to install!

- ★ `net/http` -- make the POST request
- ★ `encoding/json` -- marshal & unmarshal

```
cd bootcamp-code/day2/go/starter
go run . # that's it
```

The Go way: **decode JSON into typed structs**, not loose maps.

THE RPC HELPER (1/2)

```
type rpcRequest struct {
    JSONRPC string `json:"jsonrpc"`
    ID       string `json:"id"`
    Method   string `json:"method"`
    Params   []any  `json:"params"`
}

type rpcResponse struct {
    Result json.RawMessage `json:"result"`
    Error  *struct {
        Message string `json:"message"`
    } `json:"error"`
}
```

THE RPC HELPER (2/2)

```
func rpc(method string, params []any,
wallet string, out any) error {
url := rpcURL
if wallet != "" { url += "wallet/" + wallet }
body, _ := json.Marshal(rpcRequest{
JSONRPC: "1.0", ID: "explorer",
Method: method, Params: params})
req, _ := http.NewRequest("POST", url,
bytes.NewReader(body))
req.SetBasicAuth(rpcUser, rpcPassword)
resp, err := http.DefaultClient.Do(req)
if err != nil { return err }
defer resp.Body.Close()
var parsed rpcResponse
json.NewDecoder(resp.Body).Decode(&parsed)
if parsed.Error != nil {
return fmt.Errorf("RPC error: %s",
parsed.Error.Message)
}
return json.Unmarshal(parsed.Result, out)
}
```

USING THE HELPER

```
// Declare exactly the fields you care about
var info struct {
    Chain string `json:"chain"`
    Blocks int    `json:"blocks"`
}

rpc("getblockchaininfo", nil, "", &info)
fmt.Println("Chain:", info.Chain)
fmt.Println("Blocks:", info.Blocks)

// Wallet balance -> decode straight into a float64
var balance float64
rpc("getbalance", nil, "alice", &balance)
fmt.Printf("Alice has %v BTC\n", balance)
```

EXERCISE SETUP

```
# Make sure Bitcoin Core is running
bitcoin-cli -regtest getblockchaininfo

# Go to the Go exercise folder
cd bootcamp-code/day2/go/starter

# No pip, no deps -- just run it
go run .
```

CHALLENGE 1: BLOCKCHAIN INFO

```
func showBlockchainInfo() error {  
    // TODO:  
    // 1. Declare a struct: Chain, Blocks, Difficulty  
    // 2. rpc("getblockchaininfo", nil, "", &info)  
    // 3. Print chain, blocks, difficulty  
    return nil  
}
```

Hint: JSON keys are `chain`, `blocks`, `difficulty`

Bitcoin Day 2

CHALLENGE 1: SOLUTION

```
func showBlockchainInfo() error {
    var info struct {
        Chain      string `json:"chain"`
        Blocks      int    `json:"blocks"`
        Difficulty  float64 `json:"difficulty"`
    }
    if err := rpc("getblockchaininfo", nil, "", &info); err != nil {
        return err
    }
    fmt.Println("=== Blockchain Info ===")
    fmt.Printf("Chain:      %s\n", info.Chain)
    fmt.Printf("Blocks:      %d\n", info.Blocks)
    fmt.Printf("Difficulty:  %v\n", info.Difficulty)
    return nil
}
```

CHALLENGE 2: WALLET BALANCE

```
func showWalletBalance(wallet string) error {  
    // TODO:  
    // 1. Load wallet, ignore error if already loaded:  
    //     _ = rpc("loadwallet", []any{wallet}, "", nil)  
    // 2. rpc("getbalance", nil, wallet, &balance)  
    // 3. Print the balance  
    return nil  
}
```

CHALLENGE 2: SOLUTION

```
func showWalletBalance(wallet string) error {
    // Wallet may already be loaded -> ignore the error
    _ = rpc("loadwallet", []any{wallet}, "", nil)

    var balance float64
    if err := rpc("getbalance", nil, wallet, &balance); err != nil {
        return err
    }
    fmt.Printf("=== Wallet: %s ===\n", wallet)
    fmt.Printf("Balance: %v BTC\n", balance)
    return nil
}
```

CHALLENGE 3: LIST TRANSACTIONS

```
func listTransactions(wallet string, count int) error {  
    // TODO:  
    // 1. Load wallet (ignore error)  
    // 2. Decode into a slice of structs:  
    //     Category, Amount, TxID, Confirmations  
    // 3. rpc("listtransactions",  
    //     []any{"*", count}, wallet, &txs)  
    // 4. Print direction, amount, txid  
    return nil  
}
```

CHALLENGE 3: SOLUTION

```
func listTransactions(wallet string, count int) error {
    _ = rpc("loadwallet", []any{wallet}, "", nil)

    var txs []struct {
        Category      string `json:"category"`
        Amount         float64 `json:"amount"`
        TxID           string `json:"txid"`
        Confirmations int `json:"confirmations"`
    }

    if err := rpc("listtransactions",
        []any{"*", count}, wallet, &txs); err != nil {
        return err
    }

    for _, tx := range txs {
        dir := "OUT"
        switch tx.Category {
        case "receive", "generate", "immature":
            dir = "IN "
        }
        fmt.Printf("%s %+8f BTC | %d confs\n",
            dir, tx.Amount, tx.Confirmations)
        fmt.Printf("      TXID: %s\n", tx.TxID)
    }
    return nil
}
```

CHALLENGE 4: DECODE TX

```
func decodeTransaction(txid string) error {  
    // TODO:  
    // 1. rpc("getrawtransaction",  
    //       []any{txid, true}, "", &tx)  
    // 2. Print inputs (Vin) and outputs (Vout)  
    return nil  
}
```

Hint: a coinbase input has a non-empty `coinbase` field

CHALLENGE 4: SOLUTION

(1/2)

Describe the JSON with nested structs:

```
func decodeTransaction(txid string) error {
    var tx struct {
        Vin []struct {
            Coinbase string `json:"coinbase"`
            TxID      string `json:"txid"`
            Vout       int    `json:"vout"`
        } `json:"vin"`
        Vout []struct {
            Value          float64 `json:"value"`
            ScriptPubKey struct {
                Address string `json:"address"`
            } `json:"scriptPubKey"`
        } `json:"vout"`
    }
    rpc("getrawtransaction", []any{txid, true}, "", &tx)
    // ...print loops on the next slide
}
```

CHALLENGE 4: SOLUTION

(2/2)

Then walk the inputs and outputs:

```
for _, vin := range tx.Vin {
    if vin.Coinbase != "" {
        fmt.Println("    COINBASE (mining reward)")
    } else {
        fmt.Printf("    From: %s...\n", vin.TxID[:20])
    }
}
for _, vout := range tx.Vout {
    fmt.Printf("    %.8f BTC -> %s\n",
        vout.Value, vout.ScriptPubKey.Address)
}
return nil
}
```

CHALLENGE 5: BLOCK DETAILS

```
func showBlock(blockhash string) error {  
    // TODO:  
    // 1. If blockhash == "":  
    //     rpc("getbestblockhash", nil, "", &blockhash)  
    // 2. rpc("getblock", []any{blockhash, 1}, "", &block)  
    // 3. Print height, hash, time, tx count  
    return nil  
}
```

CHALLENGE 5: SOLUTION

```
func showBlock(blockhash string) error {
    if blockhash == "" {
        rpc("getbestblockhash", nil, "", &blockhash)
    }
    var block struct {
        Height int    `json:"height"`
        Hash    string   `json:"hash"`
        Time    int64    `json:"time"`
        NTx     int      `json:"nTx"`
        Tx      []string `json:"tx"`
    }
    if err := rpc("getblock",
        []any{blockhash, 1}, "", &block); err != nil {
        return err
    }
    fmt.Printf("=== Block #%d ===\n", block.Height)
    fmt.Printf("Hash: %s...\n", block.Hash[:32])
    fmt.Printf("Time: %d\n", block.Time)
    fmt.Printf("Transactions: %d\n", block.NTx)
    return nil
}
```

GO VS PYTHON: WHAT CHANGED

- ★ **Typed structs** instead of dict lookups
(`info.Chain`, not `info['chain']`)
- ★ **Struct tags** map JSON keys to fields
- ★ **Errors are values** -- return `error`, check it
- ★ **No** `pip` -- the standard library is enough
- ★ Same RPC calls, same Bitcoin Core

BONUS: 2-OF-2 MULTISIG

Why Multisig matters for Lightning:

- ★ Lightning channels use **2-of-2 multisig**
- ★ Both Alice AND Bob must sign to spend
- ★ Neither can steal funds alone
- ★ Foundation of trustless payment channels!

Exercise: `bootcamp-code/day2/03_multisig.sh`

(pure `bitcoin-cli` -- same for every language)

DAY 2 SUMMARY

You learned:

- ★ JSON-RPC communication with Bitcoin Core
- ★ Go `net/http` + `encoding/json` (stdlib only)
- ★ Decoding JSON into typed structs
- ★ Building a Bitcoin explorer from scratch
- ★ Transaction structure (inputs, outputs)
- ★ Block structure (header, transactions)
- ★ Multisig basics (prep for Lightning!)

Bitcoin Day 2

Tomorrow: Lightning Network!

HOMework

- ★ Complete all 5 explorer challenges
- ★ Try the multisig bonus exercise
- ★ Add a new feature to your explorer:
- ★ Show UTXO set for a wallet
- ★ Calculate total fees in a block
- ★ Search transactions by address

Solution: `day2/go/solution/`